



UNIVERZITET U NIŠU — ELEKTRONSKI FAKULTET

Horizontalno particionisanje podataka (Sharding) u savremenim DBMS sistemima

Seminarski rad — Sistemi za upravljanje bazama podataka

Student: **Katarina Adamović, br. ind. 2173**

Mentor: **Prof. dr Aleksandar Stanimirović**

Niš, 2026

Sadržaj

- 1 Uvod
- 2 Skaliranje baza podataka
- 3 Partitionisanje i sharding
- 4 Arhitektura shardovanih sistema
- 5 Strategije raspodele podataka
- 6 Izazovi horizontalne raspodele
- 7 Sharding u savremenim DBMS sistemima
- 8 Praktična implementacija (PostgreSQL)
- 9 Prednosti, nedostaci i preporuke
- 10 Zaključak

Uvod — zašto je sharding neophodan

Količina podataka raste eksponencijalno — društvene mreže, e-trgovina, IoT i finansijske platforme svakodnevno generišu terabajte novih zapisa. Baza na jednom serveru brzo dostiže granice kapaciteta, propusnosti diska, memorije i broja upita u jedinici vremena.



Problem

Jedan server ima fizičke granice kapaciteta i propusnosti.



Rešenje

Sharding — deljenje baze na šardove raspoređene na više servera.



Cilj rada

Sistematski prikaz koncepta, arhitekture i praktične realizacije u PostgreSQL-u.

Vertikalno vs. horizontalno skaliranje



Vertikalno (scale-up)

- Jačanje jednog servera (CPU, RAM, disk)
- Jednostavno — baza ostaje jedna
- Cena raste nelinearno
- Fizičke granice jedne mašine
- Jedinstvena tačka otkaza



Horizontalno (scale-out)

- Dodavanje novih, jeftinijih servera
- Kapacitet raste približno linearno
- Otkaz čvora je izolovan
- Praktično neograničen rast
- Cena: veća složenost (rutiranje, distribucija)

Replikacija, particionisanje i sharding



Replikacija

Iste podatke čuva na više čvorova. Rešava dostupnost i čitanje — ne i kapacitet ili pisanje.



Particionisanje

Deli podatke na disjunktne delove. Skalira kapacitet i propusnost pisanja.



Sharding

Particionisanje podignuto na nivo distribuiranog sistema — particije žive na nezavisnim serverima.

Ključna distinkcija

Particionisanje — deljenje tabele na fizičke delove unutar JEDNE instance DBMS-a (bolja organizacija, partition pruning).

Sharding — raspodela particija preko VIŠE nezavisnih servera, radi skaliranja kapaciteta i propusnosti izvan granica jedne mašine.

Arhitektura shardovanih sistema

Ključ raspodele (shard key)

- Visoka kardinalnost — dovoljno različitih vrednosti
- Ravnomerna raspodela — bez „vrućih tačaka“
- Usklađenost sa obrascima pristupa
- Stabilnost — vrednost se ne menja

Sloj rutiranja (3 obrasca)

- Rutiranje u aplikaciji — logika u aplikativnom kodu
- Posrednički (proxy) sloj — npr. mongos, VTGate
- Koordinatorski čvor — npr. Citus



Kolokacija (co-location)

Tabele koje se često spajaju treba shardovati po istom ključu — povezani redovi završe na istom čvoru, pa se spajanja izvršavaju lokalno, bez mrežne komunikacije (npr. tenant_id u multi-tenant sistemima).

Strategije horizontalne raspodele podataka

Strategija	Prednost	Nedostatak	Predstavnici
Po opsegu (range)	Efikasni opsegovni upiti	Vruće tačke kod monotonih ključeva	Spanner, CockroachDB, HBase
Heširanjem (hash)	Ravnomerna raspodela	Opsegovni upiti pogađaju sve šardove	Citus, MongoDB, YugabyteDB
Konzistentno heširanje	Minimalno premeštanje pri rebalansiranju	Složenija implementacija	Cassandra, DynamoDB, Riak
Preko imenika (directory)	Potpuna fleksibilnost	Dodatni skok; kritična komponenta	Prilagođena rešenja, Vitess (delom)
Geografska	Niska latencija, regulativa (GDPR)	Neravnomerno opterećenje regiona	Spanner, CockroachDB (multi-region)

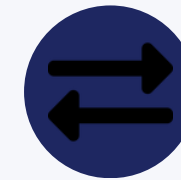
Sistemi u praksi često kombinuju strategije — npr. Cassandra: heš particioni ključ + sortirane klaster-kolone unutar particije.

Izazovi horizontalne raspodele podataka



Distribuirani upiti

Ciljani upit pogađa jedan šard. Upit bez ključa → scatter-gather na svim šardovima. Cross-shard JOIN zahteva data shuffling.



Distribuirane transakcije

2PC (two-phase commit) garantuje atomičnost ali unosi dodatne mrežne runde i brave.



CAP teorema

U prisustvu particije mreže, sistem bira između konzistentnosti (C) i dostupnosti (A).



Globalna jedinstvenost ID

Auto-increment ne radi preko čvorova — rešenja: UUID, kompozitni ključevi, Snowflake ID.

Rebalansiranje i operativna složenost

Rebalansiranje / resharding

- Premestiti minimalnu količinu podataka
- Sistem ostaje dostupan (online rebalancing)
- Kontrolisano opterećenje premeštanja
- Promena ključa raspodele je najskuplja operacija

Operativni teret

- Rezervne kopije preko više čvorova
- Koordinisane migracije šeme
- Nadzor ravnomernosti raspodele
- Složenije testiranje i razvoj



Preporuka: sharding uvoditi tek kada su iscrpljene jednostavnije opcije — optimizacija upita, vertikalno skaliranje, keširanje, replike za čitanje.

Sharding u savremenim DBMS sistemima

Sistem	Model	Strategija	Transakcije preko šardova
MongoDB	Dokument	Opseg ili heš (chunks)	Da (od v4.2, 2PC)
Cassandra	Wide-column	Konzistentno heširanje	Ne (eventual consistency)
Vitess / MySQL	Relacioni	Heš / opseg (VSchema)	Ograničeno (2PC opciono)
Citus / PostgreSQL	Relacioni	Heš; referentne tabele	Da (2PC)
Spanner / CockroachDB	Relacioni (NewSQL)	Dinamički opsezi	Da (Paxos/Raft)

Jasna evolutivna putanja: od ručno shardovanih relacionih baza, preko NoSQL sistema, do NewSQL rešenja sa transparentnim shardingom.

Tri koraka — od jedne instance do klastera

01

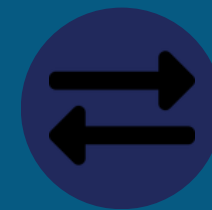


Deklarativno particionisanje

Jedna PostgreSQL instanca, 4 heš particije

1.000.000 redova

02

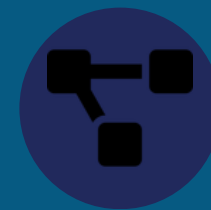


postgres_fdw (2 servera)

Particije kao strane tabele na udaljenim serverima

1.000.000 redova

03



Citus klaster

Koordinator + 2 radna čvora, 32 šarda po tabeli

10.000.000 redova



Korak 1 — partitionisanje unutar jedne instance

Postavka eksperimenta

- Obična tabela porudzbina_obicna — 1.000.000 redova, bez particija
- Partitionisana tabela porudzbina — PARTITION BY HASH (kupac_id)
- 4 particije (p0–p3), MODULUS 4, isti broj redova (1.000.000)
- Tri tipa upita: ciljani (kupac_id=42), bez ključa (status='NOVA'), globalna agregacija

```
CREATE TABLE porudzbina (  
  id bigint, kupac_id bigint,  
  datum timestamptz, iznos numeric,  
  ...  
) PARTITION BY HASH (kupac_id);
```

```
CREATE TABLE porudzbina_p0  
  PARTITION OF porudzbina  
  FOR VALUES WITH  
  (MODULUS 4, REMAINDER 0);
```

4 particije ukupno: p0, p1, p2, p3 (MODULUS 4, REMAINDER 0-3)

Korak 1 — rezultati merenja (EXPLAIN ANALYZE)

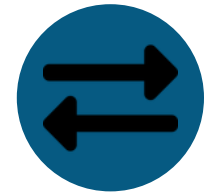
Upit	Obična tabela	Particionisana (4)	Efekat
Ciljani upit (kupac_id = 42)	104,5 ms	38,4 ms	≈2,7x brže
Bez ključa (status = 'NOVA')	144,5 ms	236,9 ms	sporije
Globalna agregacija (sum nad celom tabelom)	129,8 ms	139,8 ms	malo sporije



Zaključak

Particionisanje unutar jedne instance NIJE opšte ubrzanje čitanja. Pošto sve particije dele isti disk i CPU, ono pomaže isključivo ciljanim upitima (eliminacija particija). Upiti bez ključa raspodele su čak sporiji zbog režijskog troška paralelnog obilaska svih particija.

Korak 2 — sharding preko mreže (postgres_fdw)



- Particije roditeljske tabele su FOREIGN TABLE, koje pokazuju na fizičke tabele na dva udaljena servera (`postgres_fdw`).
Heš raspodela po `kupac_id` (MODULUS 2) — svaki server nezavisno čuva ~pola podataka.
Aplikacija i dalje radi samo sa jednom logičkom tabelom porudzbina na koordinatoru.

Korak 2 — cena mreže: rezultati

499.971 : 500.029

raspodela redova shard1 : shard2
(razlika 0,006%)

26,1 ms

ciljani upit
(kupac_id = 42)

1.982 ms

upit bez ključa
(scatter-gather, oba servera)

Vreme umetanja: 2 min 15 sek za milion redova preko mreže (naspram par sekundi lokalno) — svaki red putuje sa koordinatora do pravog servera.



Zaključak

postgres_fdw je funkcionalan, ravnomeran sharding (razlika < 0,01%). Cena je vidljiva samo u dva mesta — sporiji upis i veća latencija — i oba potiču isključivo od mrežne komunikacije. Upit bez ključa raspodele je >75x sporiji od ciljanog: direktna potvrda da aplikacija mora znati ključ raspodele.

Korak 3 — pravi klaster: Citus



Postavka

- Docker Compose klaster: master (koordinator) + worker1 + worker2
- `create_distributed_table('porudzbina', 'kupac_id')` — 32 šarda po tabeli
- Kolokacija: `porudzbina` i `kupac` dele isti ključ → `colocate_with`
- Referentna tabela `status_sifra` replicirana na oba čvora
- 100.000 kupaca, 10.000.000 porudžbina

master (koordinator)

worker1

16 šardova
441 MB

worker2

16 šardova
444 MB

Ravnomerna raspodela: 32 šarda podeljena tačno 16/16 između čvorova

Korak 3 — pravi paralelizam: rezultati

21,35 ms

JOIN kupac + porudzbina
(kolokacija, lokalno spajanje)

16 / 16

šardova po čvoru
(worker1 / worker2)

1 po čvoru

kopija referentne tabele
status_sifra na svakom radniku

Ključna razlika u odnosu na korake 1 i 2: šardovi imaju sopstveni CPU, memoriju i disk. Globalna agregacija bez ključa raspodele se ovde *stvarno paralelizuje* — koordinator šalje deo upita na worker1 i worker2 istovremeno, umesto da samo trpi režijski trošak kao u koraku 1.



Zaključak

Sharding preko nezavisnih servera pretvara scatter-gather iz mane (korak 1) u prednost: pravi paralelizam. Kolokacija i referentne tabele eliminišu mrežnu komunikaciju za najčešće operacije (spajanja).

Šta smo zaključili iz sva tri eksperimenta

	1. Particionisanje (1 instanca)	2. postgres_fdw (2 servera)	3. Citus (klaster)
Deljeni resursi?	Da — isti disk/CPU	Ne — nezavisni serveri	Ne — nezavisni čvorovi
Ciljani upit	2,7x brže	26 ms (mreža)	21 ms + kolokacija
Upit bez ključa	sporije (režija)	1982 ms (75x)	paralelizovano
Glavna pouka	Korist samo za ciljane upite	Ravnomerno, ali mreža ima cenu	Kolokacija čini spajanja lokalnim

Zajednička nit: ključ raspodele u uslovu upita je jedini faktor koji stvarno određuje performanse.

Zaključak

- **Sharding nije univerzalno rešenje već inženjerski kompromis**
- Uspeh presudno zavisi od izbora ključa raspodele
- Strategije (opseg, heš, konzistentno heširanje) nude različite kompromise
- Praktični eksperimenti potvrđuju teoriju: particionisanje $\neq$ sharding, kolokacija eliminiše mrežnu komunikaciju
- Trend: NewSQL sistemi čine horizontalnu raspodelu sve transparentnijom

Hvala na pažnji